

A memory that outlives the robot

Spatial Memory for Emergency Robots
IEEE RAS x IEEE AESS Tunisia Section - TSYP14 technical challenge

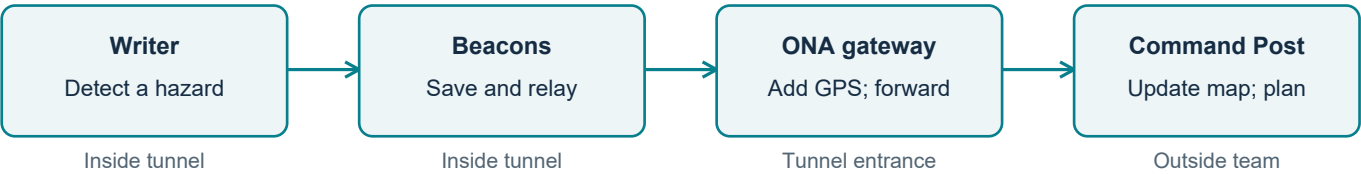
1. Why the project exists; what works now

An emergency robot can discover hazards in a tunnel where GPS and direct outside communication are unavailable. If it fails, its private knowledge may be lost. The Living Map saves selected findings in small radio beacons so a later robot can act without restarting exploration from zero.

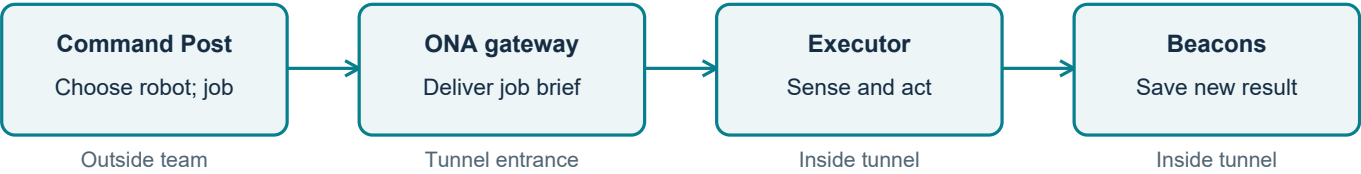
Implemented now: autonomous tunnel exploration; FIRE/GAS detection; beacon storage and radio relays; local-to-GPS conversion; outside mission planning; response robots; and recovery after explorer failure or a blocked route. These run in the Python/Pygame simulation. **Next steps:** physical robots, real radio and sensors, saved data across restarts, and SLAM. No physical prototype or field accuracy is claimed.

2. How information becomes a mission

1 SAVE A FINDING AND SEND IT OUTSIDE



2 SEND A JOB IN; WRITE BACK WHAT HAPPENED



The new result follows flow 1 again. Repeated boxes show the same components.
Figure 1. Read each row left to right. The Command Post chooses the work. The ONA carries information and converts positions. A robot's new observation returns through the same beacon network.

Component	Plain-language role and implemented responsibility
Writer	Explorer: discovers its own map, detects events, chooses what to save and where to place a beacon.
Beacon	Radio marker: stores records and relays packets toward the entrance, with buffering and retries.
ONA	Entrance gateway: validates received data, converts coordinates, forwards findings and delivers briefs.
Command Post	Outside team: maintains the Living Map, prioritizes work and chooses a capable, available robot.
Executor	Response robot: follows inherited information, senses locally, acts, updates memory and returns.

Robots communicate with beacons and the ONA through RobotLink; only the ONA connects to the Command Post. Each robot navigates with its own discovered map. The display can show the full simulation, but robots cannot use that hidden knowledge. Tests check these boundaries. [1, 2]

Explore, decide and preserve

3. Autonomous Writer and event decisions

The Writer explores the edge of known space, called a **frontier**. A* route planning uses only the map it has discovered. Local sensors reveal walls and hazards as it moves. A fixed seed repeats the same choices; the route itself is not scripted. The Writer budgets time to return to the entrance. An injected failure tests what happens if it cannot return.

Position is measured in local metres from the entrance. The demos use ideal simulated movement tracking, not SLAM or calibrated physical sensors. FIRE/GAS findings are saved at severity 2 or above and confidence 50% or above. Nearby same-type findings within two horizontal/vertical grid steps are suppressed. These are test settings, not measured detection thresholds.

The Writer reuses a nearby beacon with space. With few beacons left, it saves them for important events. If a finding cannot reach the entrance, it moves back toward a connection and places relay beacons using locally measured signal strength. The log explains each decision. [2]

4. Beacon type, stored data and radio messages

Current type: a programmable, simulated **radio memory-and-relay beacon** (BeaconNode). An event beacon stores findings; a relay-only beacon bridges a gap and may hold none. Both use the same node design. Default capacity is four findings per beacon. A beacon's ID identifies the host; a memory ID identifies one finding.

Part	Fields	Bytes
Beacon packet	Host ID; event ID; x/y float32; timestamp uint32; severity; confidence; battery; CRC	18
Memory extension	Memory ID uint16; version; lifecycle; passage state; latest observation time; source node	10
Mesh envelope	Type; origin; sequence; TTL; hop count; link source/destination/gradient; length; outer CRC	11
Complete MEMORY frame	18-byte packet + 10-byte extension + 11-byte envelope	39

ONE TRANSMITTED FINDING = 39 BYTES

Mesh header: 10 B Routing	Finding: 18 B What / where / when	Memory: 10 B State / version	
------------------------------	--------------------------------------	---------------------------------	--

Byte order, left to right; base finding includes its own CRC. Outer CRC: 1 B

Figure 2. Header and outer checksum together form the 11-byte mesh envelope. GPS is added later by the ONA; it is absent from this frame.

Each radio hop chooses a neighbour nearer the entrance, waits for acknowledgement and retries if needed. Nodes keep undelivered packets, ignore duplicates and limit the number of hops (TTL). CRC checks damaged bytes. The guided Details view shows actual received bytes, packet size and relay history. [3]

Wire detail: the complete finding is 39 bytes. CRC uses MSB-first polynomial 0x31, initial/final XOR zero; test bytes 123456789 give 0xA2. Identifiers and versions have explicit range limits; exhaustion raises an error instead of wrapping.

Planned physical type: Arduino Nano + SX1278/Ra-02 LoRa radio + battery. Real radio transport and durable storage are not implemented. Range, interference, power use and battery life still need measurement. Storage and queues are finite; a simulated acknowledgement proves reception, not a flash-memory write. [2]

Keep memory consistent and useful

5. Where the data lives; how it stays trustworthy

Inside each beacon, **BeaconNode.records** holds the event packet and memory extension in RAM. The ONA delivers copies to the Command Post's Living Map. Each finding keeps its latest observation time, confidence, reporting robot and host beacon. The outside map lowers confidence as a report ages, without declaring the hazard gone. An Executor checks the site and saves a fresh observation.

Record state	Meaning for the next mission
UNVERIFIED	A saved report exists, but a response robot has not checked it.
VERIFIED / ACTIVE	The report was confirmed; ACTIVE means the hazard still exists.
CLEARED	The event or requested obstruction has been resolved.
CONTRADICTED	Local sensing did not find the remembered event.
ESCALATED	The problem persists and requires further help.

Reports of the same event within 0.3 m share **one main record** (the canonical record). Other IDs remain valid alternate names (aliases). Versions are tracked separately for each incoming ID: a repeated packet is not another confirming observation. Older reports cannot reopen a cleared event; at equal times, a resolved state beats an unverified one. Accepted changes advance the main record's revision. History keeps incoming IDs/versions, and the display follows a finding moved to another host. [2, 4]

6. ONA: convert positions and carry information

The entrance gateway checks packets, removes duplicates, adds GPS positions and forwards findings. It holds failed outside deliveries for retry. Executors collect job instructions from its mailbox before entering. The Command Post plans the jobs. A software connection stands in for future wireless/satellite communication; no real distant link has been tested.

Local (0, 0) is the entrance. Heading is counter-clockwise from east. First rotate x/y into east/north metres, then approximate GPS near the entrance. In the equations, theta is heading in radians; ref_lat_rad is reference latitude converted to radians. GPS output is in degrees. [3]

```
east = x cos(theta) - y sin(theta)
north = x sin(theta) + y cos(theta)
latitude = ref_lat + north / 111111
longitude = ref_lon + east / (111111 cos(ref_lat_rad))
```

Recorded example: the normal demo's fire finding at local (2.0, -3.0) m becomes GPS (36.8064730, 10.1815225). GPS is added by the ONA, not transmitted in the 39-byte MEMORY frame. Coordinate tests check axes, headings and round trips. Seven-decimal output precision is not a measured positioning accuracy. [4]

What survives a failure

A Writer failure leaves beacon findings available while the simulation runs. Closing the application loses this RAM state. JSON exports let a reviewer inspect results; they do not restore a database. Restart persistence and SLAM remain next steps.

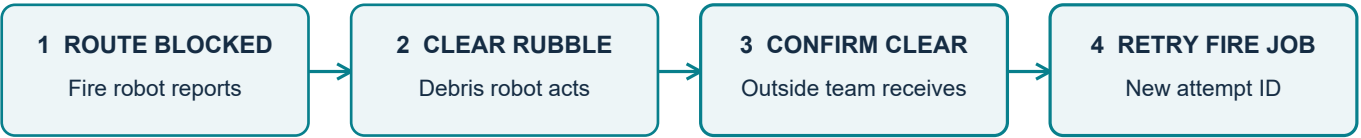
Finish the mission and recover

7. Planning and Executor behavior

The Command Post chooses urgent, believable and reachable jobs using severity, confidence, report age, hazard type, record state and travel cost. It chooses an available robot with suitable equipment, then prefers more battery and fewer previous jobs. The job brief includes the target, known beacon positions, hazards and instructions for a blocked route. These priority rules are test settings.

The Executor uses those hints, builds its own local map and changes route when it senses an obstacle. It can put out a simulated fire or investigate gas. It checks the result, updates a beacon, reports the outcome and returns home. Gas investigation leaves the record ACTIVE: the job is finished, but the gas hazard remains. [2]

8. Blocked route: clear it, then retry



The original job waits until clearance reaches the Command Post.

Finish only when the hazard record is updated and the response robots return home.
If delivery, clearance or return fails, the demo stops with a reason; it cannot claim success.

Figure 3. Rubble appears after exploration. The first fire attempt reports a blockage. A DEBRIS robot clears the requested obstruction. Only after the outside team receives clearance does the original fire job retry. Another unrelated blockage may remain recorded under the on-demand policy. [4]

Each dispatch gets a new attempt number, including a retry. A late result for an old attempt cannot finish a newer assignment or make its robot available. Old status also cannot roll back a newer robot state. Each outcome counts once, and each mission appears once in the queue. This prevents duplicate retries. Attempt IDs are positive 16-bit values unique within the running process. Old nine-byte STATUS payloads still decode; the new eleven-byte format adds the attempt ID. Old messages cannot alter a modern assignment. [3, 4]

Failure or uncertainty	Demonstrated response / honest limit
Writer fails after saving	Beacon findings survive; response robots still complete their jobs.
Rubble blocks a route	Blocked job requests clearance, then retries once as a new attempt.
Event no longer present	Executor sensing writes CONTRADICTED, rather than trusting the old report.
Relay interrupted	A stored record remains during disconnection and arrives after reconnection.
Missing delivery, fault, low battery	Unfinished acceptance checks time out with a readable stop reason.
Conflicting final copies / extra retry	All final checks are required; a failed check cannot display success.

Eleven injected failures exercise early explorer loss, outside delivery loss, missing briefs, unavailable clearance, fault, low battery, stranding, conflicting copies, extra retry, missing return and runtime error. The controller stops and cannot silently resume; R resets it. These tests show correct failure reporting, not recovery from every possible fault. [4]

Validation and actual results

9. What we checked

The recorded full suite has **399 passing tests**, zero failures, errors or skips (100.080 s). It used Python 3.13.5, pytest 9.1.1 and Pygame 2.6.1. Tests cover alternate IDs, duplicate/old reports, moved hosts, late/repeated outcomes, retry uniqueness, both status formats and range limits. Runtime source hashes still match the tested files. [4]

Guided scenario (seed 42)	Outcome	Final tick
normal	All acceptance checks PASS	3469
writer-failure	All acceptance checks PASS	2329
debris-recovery	All acceptance checks PASS	5249

A demo passes only when expected records arrive, dispatched jobs finish, response robots return and become available, queues contain no duplicate or eligible pending work, and beacon copies agree with the outside map. The network runs for sixty more stable simulation steps to catch late updates. Missing results stop the demo with a reason.

Automated Windows input checked phase advance, pause/resume, reset, scenario switching, Jobs/Findings, Details, selection and scrolling. Captures were inspected for readable, unclipped content. This is automated UI verification, not a human walkthrough.

10. Does inherited memory help? Six-seed comparison

Seed	Travel: memory	Travel: baseline	Ticks: memory	Ticks: baseline
42	36	74	938	1538
43	40	70	1268	1593
44	35	87	968	1783
45	31	31	998	1013
46	37	71	908	1447
47	19	19	638	647

Travel counts grid cells on the outward trip; ticks count simulation steps, not real seconds. Baseline means the robot starts without inherited memory. Each pair uses the same arena and debris; all twelve runs reached their targets. New debris is unknown to both robots. Seeds 45 and 47 have equal travel, so memory does not improve every layout. [4]

11. Old reports, missing events and broken relays

The aging check reduced live confidence from 39% to 5% after a 1200-second simulated-time jump while retaining UNVERIFIED. Local Executor intervention refreshed the record to 91% confidence and CLEARED. A separate missing-event test produced CONTRADICTED through sensing. The four-hop relay fixture retained the deepest record during an outage and delivered it after reconnection; the seed-5 global demo formed a seven-hop chain. [4]

Reproducible identity. Base a7db087ff6c2 plus local changes. Full revision, source fingerprint and per-file SHA-256 hashes: audit/phase1/verification.json. [4]

Technical synthesis and development

12. Phase 1 technical contribution

Phase 1 establishes a complete information loop in simulation: the Writer explores and preserves FIRE/GAS findings; beacons store and relay them; the ONA converts positions and exchanges information with the Command Post; and Executors use that knowledge to complete missions and update the map. The demonstrated recovery cases preserve useful findings after Writer failure and restore access when debris blocks a mission. [2, 4]

The central contribution is the selective sharing of operational knowledge between successive robots. Versioned observations, report aging and local verification keep that knowledge useful as the environment changes. The three diagrams describe the architecture, transmitted data and recovery sequence; the results on page 5 establish the current software validation. These results concern the modeled tunnel and simulated sensing, movement and radio.

13. Physical integration and experimental validation

The proposed hardware stage retains the existing division of responsibilities. ESP32 controllers are planned for two robots and the ONA; Arduino Nano controllers with SX1278/Ra-02 radios are planned for the beacons. Component quantities and wiring will first be reconciled, followed by power, motor, encoder, event-sensor and beacon-deployment tests. Physical assembly and calibration remain to be completed. [2]

Radio integration will implement the current packet format over LoRa and add storage that survives a beacon restart. Tests will examine delivery through several relays, interruption and reconnection, and retention of saved findings. The complete missions will then be repeated on physical robots, recording delivery rate, range, battery use, detection performance and position error. Successful validation requires confirmed observations, completed dispatched jobs and robot return, as in the simulation.

14. Localization and mapping with SLAM

A further development is **Simultaneous Localization and Mapping (SLAM)**: estimating the robot's position while building a map in the GPS-denied tunnel. The present demonstrations use ideal simulated movement tracking; SLAM is not yet implemented. Candidate sensing will combine lidar or camera observations with encoders and an inertial sensor. Sensor suitability and computing requirements will be evaluated before selecting the hardware.

An initial mapping prototype will be compared with a known layout to measure position drift and map consistency. Its position and map outputs will then feed the existing navigation interfaces. SLAM will support geometric localization, while beacons will continue preserving hazard reports and mission observations. This extension therefore complements the Living Map's operational memory.

15. Reproducibility and supporting material

The repository README provides installation, guided controls and headless execution instructions. The normal demonstration can be launched from the repository root using the configured Python environment:

```
python -B run_simulation.py --guided --scenario normal --seed 42
```

The same entry point supports writer-failure and debris-recovery. Source hashes, automated results and captures are retained in audit/phase1/. Execution was verified on Windows; README also provides macOS/Linux setup instructions. [4]

References. [1] TSYP14, The Living Map: Spatial Memory for Emergency Robots, challenge brief, pp. 1-3. [2] Project implementation; docs/architecture/system-architecture.md; hardware/README.md. [3] common/protocol.py; common/coordinates.py; docs/communication/beacon-protocol.md. [4] audit/phase1/{verification.json, pytest.xml, scenarios.json, supplemental.json, ui/}; docs/phase1_demo.md; docs/phase1_traceability.md. Repository: github.com/abdelkqder/The-Living-Map-TSYP14.